

Desarrollo de el *inflight-software* para la OBC secundaria del Satélite Quetzal-2



Autor: Flavio André Galán Donis

Curso: Diseño e Innovación

Guatemala, 15 de abril de 2026

Desarrollo de el *inflight-software* para la OBC secundaria del Satélite Quetzal-2

Flavio Galan

Universidad del Valle de Guatemala

gal22386@uvg.edu.gt

Resumen/Abstract

El Quetzal-2 es un nanosatélite CubeSat 2U desarrollado como proyecto académico por estudiantes y *staff* UVG el cual pone a prueba una *On-Board Computer* (OBC) diseñada localmente que ejecuta una inteligencia artificial capaz de identificar nubes. Este nanosatélite cuenta con una computadora a bordo compuesta de por dos unidades de procesamiento, una *of-the-shelf* de GomSpace y otra desarrollada localmente utilizando Portentas H7 de Arduino.

Las OBC son sistemas vitales dentro de una misión espacial puesto que gestionan todos los subsistemas del satélite así como la comunicación entre estos. Este proyecto se centra en las bases de esta OBC local a nivel de *software*, sentando las bases para los distintos modos de operación que necesitará la misión, la toma de fotografías y la interoperabilidad entre las dos unidades de procesamiento que componen la OBC completa. El impacto de esta OBC desarrollada localmente representa un gran punto de inicio para próximas misiones espaciales, bajando aún más la barrera económica de entrada para desarrollar misiones espaciales con respecto a soluciones *of-the-shelf*, puesto que demuestra que no es necesario el uso de hardware altamente costoso con muchas más protecciones para llevar a cabo una misión exitosa.

INTRODUCCIÓN

El Quetzal-2 es un proyecto académico desarrollado por estudiantes y personal UVG la cual pone a prueba una computadora a bordo (OBC por sus siglas en inglés) diseñada en UVG capaz de ejecutar un modelo de inteligencia artificial para identificar nubes en imágenes satelitales [1]. Este proyecto es importante no solo porque representa un avance tecnológico en Guatemala, sino también por su impacto dentro de la juventud del país, inspirando a futuros científicos, ingenieros e innovadores [2].

La misión espacial es un CubeSat 2U, 10x20x10cm y con un peso máximo de 4kg [3] y cuenta con 4 objetivos específicos que busca completar, llamadas cargas útiles (*Payloads* en inglés). Para este proyecto nos interesa específicamente *Payload MILO*, el subsistema encargado del reconocimiento de nubosidad y la toma de fotografías en el espacio [1].

Las OBC cumplen un papel clave dentro de los subsistemas del satélite puesto que son las encargadas de manejar todas las tareas, intercambio de información entre

módulos y la colecta de información sobre los demás subsistemas (*housekeeping*) antes de la conexión con la estación en tierra [4].

Algunos de los subsistemas que debe manejar una OBC son:

- Un sistema de poder (EPS por sus siglas en inglés).
- Un sistema que determina la altitud (ADCS por sus siglas en inglés).
- Un sistema de comunicación utilizando radiofrecuencias.
- Una o más cargas útiles, por ejemplo una cámara o transmisor de señales junto con su controlador.

La inicialización, intercomunicación y gestión de todos estos subsistemas recae sobre la OBC [5]. Debido a las altas limitaciones energéticas en satélites de esta escala se debe tener especial cuidado con el presupuesto de energía que se le puede brindar a cada uno de los subsistemas. De esta necesidad nacen los modos de operación, los cuáles restringen las funcionalidades del satélite según la tarea para la que se quiere que se especialice en ese momento.

Este trabajo profesional busca desarrollar un sistema de *software* para la OBC diseñada localmente (*in-house*) capaz de operar la carga útil MILO, gestionar modos de operación y ejecutar un mecanismo de handover con la computadora principal, validado en un entorno controlado terrestre. Para el desarrollo de este sistema se utilizará una metodología propia del laboratorio espacial UVG, basada en la metodología de ingeniería de sistemas de la NASA [6].

ANTECEDENTES

1. Investigacion preliminar.

OBJETIVOS

Desarrollar localmente las bases de un *inflight software* para la OBC diseñada localmente del Quetzal-2, capaz de operar la carga útil MILO, gestionar modos de operación y ejecutar un mecanismo de handover con la OBC principal, validado en un entorno controlado terrestre.

1. Integrar el módulo de cámara al sistema de *software* de la computadora a bordo, logrando la captura y transmisión de al menos una secuencia de imágenes desde la carga útil MILO hacia la computadora a bordo desarrollada localmente, sin errores de comunicación en pruebas controladas.
2. Desarrollar el sistema base de gestión de modos de operación del *software*, implementando al menos tres modos (Arranque, Toma de fotografía y Nominal preliminar), validando transiciones correctas entre los modos de Toma de fotografía y Nominal preliminar sin pérdida de funcionalidad.
3. Desarrollar un Producto Mínimo Viable (MVP por sus siglas en inglés) del sistema de handover entre la OBC principal y la OBC diseñada localmente, que permita transferir y recuperar el control de la carga útil MILO mediante comandos definidos y verificados en pruebas de integración.

JUSTIFICACIÓN

La misión espacial de Quetzal-2 tiene como misión principal probar una computadora a bordo (diseñada en UVG) que sea capaz de ejecutar un modelo de inteligencia artificial para identificar nubes en imágenes satelitales [1]. Al ser una misión académica y no un proyecto de gobierno o una multinacional como Tesla, los recursos económicos son relativamente limitados. Esto genera una necesidad de reducir costos en donde sea posible y por lo tanto, un *software* diseñado *inhouse* adaptado a las necesidades específicas de la misión pero con oportunidad de ser extendido a otras misiones futuras capaz de ejecutarse dentro de componentes más baratos suena muy atractivo, dando origen a este proyecto.

Otra limitante dentro de estos sistemas es el bajo presupuesto de energía con el que se cuenta. En general, esto afecta la potencia de los microcontroladores que se pueden utilizar, pues se tiene que balancear disponibilidad del sistema y potencia [4]. Por lo tanto, esta clase de sistemas espaciales generalmente utilizan sistemas operativos en tiempo real (RTOS por sus siglas en inglés) debido a que ofrecen un mayor control sobre el orquestamiento de tareas permitiendo así (de forma indirecta) un mayor control sobre el uso de energía.

Para este proyecto en específico se consideraron dos opciones de sistemas operativos a utilizar, ambos RTOS y *open source*: RODOS y FreeRTOS.

1. RODOS: Es un sistema operativo basado en el principio de cliente-servidor. Su principal fortaleza es un diseño sencillo y modular que permite su extensibilidad [7].
2. FreeRTOS: Es un sistema operativo cuyos fuertes principales son su facilidad de uso y el alto soporte que tiene tanto en comunidad como en arquitecturas de procesador, en especial, que ya soporta la Portenta H7 [8].

Para este proyecto se utiliza FreeRTOS como sistema operativo base para el *inflight software*. Siendo el soporte para la PortentaH7 su principal ventaja respecto a RODOS.

METODOLOGÍA

Se seguirá una metodología kanban de trabajo, con columnas que representan los estados:

1. Listos
2. En progreso
3. Completado

Además para la documentación y organización de los módulos se utilizará una metodología propia del laboratorio pero que tiene sus bases en *Systems Engineering*, una metodología utilizada por la *National AeroSpace Agency* (NASA) para sus proyectos [6].

La modalidad será en su mayoría presencial debido a que para realizar pruebas se necesita hardware específico que se encuentra disponible solamente dentro del laboratorio aeroespacial UVG. Sin embargo, el desarrollo de *software* tiene la gran ventaja que puede ser trabajado de forma remota también, por lo que ocasionalmente se tendrán sesiones asíncronas remotas de desarrollo.

PLAN DE TRABAJO

Tarea	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Fase 1 · Integración con la carga útil MILO																		
Pruebas de comunicación entre subsistemas																		
Definición de comandos sobre MILO																		
Desarrollo de comando de toma de fotografía																		
Desarrollo de envío de fotografía																		
Fase 2 · Sistema de gestión de modos de operación																		
Diseño de arquitectura de la OBC																		
Implementación del <i>runtime environment</i> de la OBC																		
Implementación del modo de arranque																		
Implementación del modo de toma de fotografía																		
Implementación del modo nominal preliminar																		
Fase 3 · MVP Sistema de <i>Handover</i>																		
Establecer comunicación entre OBC principal y OBC secundario																		
Definición de comandos de comunicación																		
Implementación del comando de toma de control																		
Implementación del comando de retiro de control																		
Implementación del retiro de control de forma automática																		
Pruebas de funcionamiento y correcciones																		

BIBLIOGRAFÍA

- [1] U. del Valle de Guatemala, «Satélite CubeSat Quetzal-2». [En línea]. Disponible en: <https://www.uvg.edu.gt/quetzal-2/>
- [2] UVG, «CubeSat». [En línea]. Disponible en: <https://www.uvg.edu.gt/cubesat/>
- [3] A. Johnstone, *CubeSat Design Specification Rev. 14.1 The CubeSat Program, Cal Poly SLO CubeSat Design Specification Cal Poly -San Luis Obispo, CA Document Classification X Public Domain*. 2022. [En línea]. Disponible en: https://static1.squarespace.com/static/5418c831e4b0fa4ecac1bacd/t/62193b7fc9e72e0053f00910/1645820809779/CDS+REV14_1+2022-02-09.pdf
- [4] T. Lwabanji, R. Wilkinson, y E. Biermann Bellville, *Development of an OnBoard Computer (OBC) for a CubeSat*. 2013. [En línea]. Disponible en: https://etd.cput.ac.za/bitstream/20.500.11838/1172/1/Lumbwe_T_Final2013.pdf
- [5] S. A. Akhtar, *Design and Development of Obc of a Pico Spacecraft*. LAP Lambert Academic Publishing, 2012.
- [6] S. R. Hirshorn, *NASA Systems Engineering Handbook*. 2007. [En línea]. Disponible en: https://www.nasa.gov/wp-content/uploads/2018/09/nasa_systems_engineering_handbook_0.pdf?emrc=4096b9
- [7] O. Schuele, M. Rafat, y V. Kossykh, «The software environment of RODOS», pp. p.1109-1118, 1996.
- [8] FreeRTOS, «FreeRTOS - Market Leading RTOS (Real Time Operating System) for Embedded Systems with Internet of Things Extensions». [En línea]. Disponible en: <https://www.freertos.org/>